Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет Компьютерного Проектирования
Кафедра инженерной психологии и эргономики
Дисциплина: Компьютерные системы и сети

ОТЧЁТ
к лабораторной работе
на тему
«ВВЕДЕНИЕ В МОДЕЛЬ OSI»

Выполнил:
ст. гр.410901
Кляус А.Б.

Проверила:
Болтак С.В.

Минск 2026

Задание 1. Ping

1. Изучите принцип работы команды ping

Команда ping отправляет icmp запрос по указанному адресу. В ответ на что, указанный адрес возвращает в свою очередь ответное сообщение.

В итоге команда показывает затраченное время на отправку запроса туда и обратно, а также указывает, были ли потеряны пакеты в ходе обмена сообщениями.

2. Отфильтруйте в Wireshark только ICMP-трафик.

Рисунок 1.1 – Фильтрация по icmp

3. Запустите в консоли команду ping над любым узлом (в том числе над узлом локальной сети).

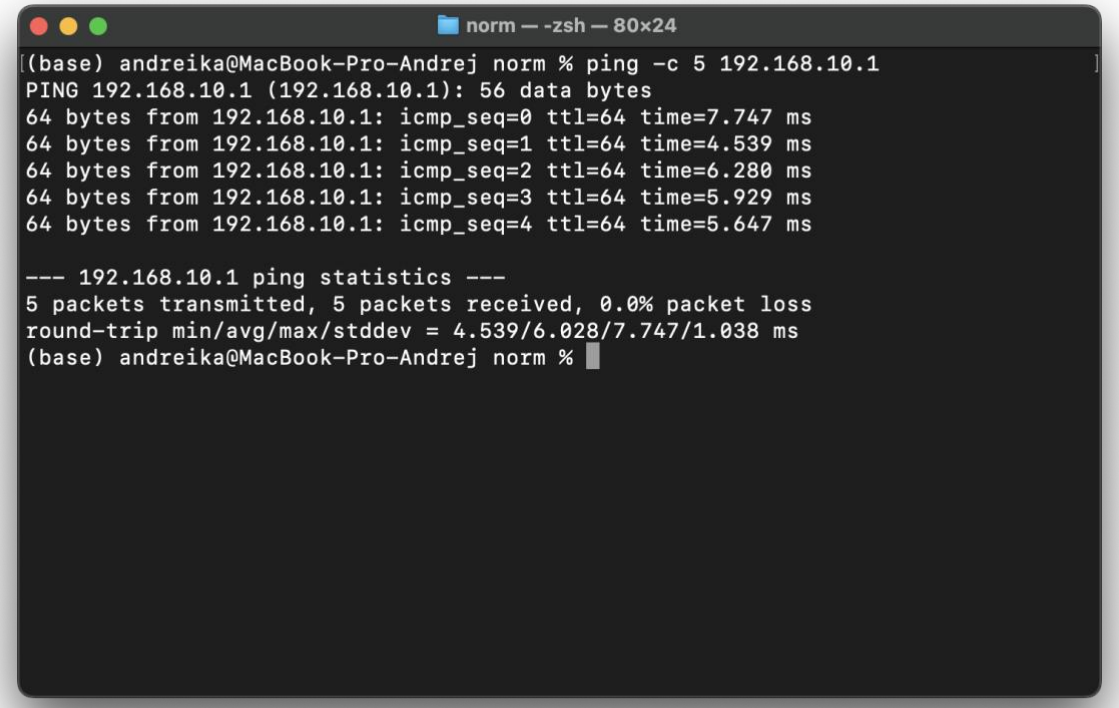


Рисунок 1.2 – Работа команды ping

4. Соотнесите и прокомментируйте вывод команды ping и перехваченные сообщения.

No.	Time	Source	Destination	Protocol	Length	Info
88	4.114532	192.168.10.174	192.168.10.1	ICMP	98	Echo (ping) request id=0x21ec, seq=0/0, ttl=64 (reply in 89)
89	4.118973	192.168.10.1	192.168.10.174	ICMP	98	Echo (ping) reply id=0x21ec, seq=0/0, ttl=64 (request in 88)
92	5.112767	192.168.10.174	192.168.10.1	ICMP	98	Echo (ping) request id=0x21ec, seq=1/256, ttl=64 (reply in 93)
93	5.116528	192.168.10.1	192.168.10.174	ICMP	98	Echo (ping) reply id=0x21ec, seq=1/256, ttl=64 (request in 92)
107	6.118294	192.168.10.174	192.168.10.1	ICMP	98	Echo (ping) request id=0x21ec, seq=2/512, ttl=64 (reply in 108)
108	6.123856	192.168.10.1	192.168.10.174	ICMP	98	Echo (ping) reply id=0x21ec, seq=2/512, ttl=64 (request in 107)
111	7.124191	192.168.10.174	192.168.10.1	ICMP	98	Echo (ping) request id=0x21ec, seq=3/768, ttl=64 (reply in 112)
112	7.129514	192.168.10.1	192.168.10.174	ICMP	98	Echo (ping) reply id=0x21ec, seq=3/768, ttl=64 (request in 111)
114	8.127433	192.168.10.174	192.168.10.1	ICMP	98	Echo (ping) request id=0x21ec, seq=4/1024, ttl=64 (reply in 115)
115	8.132489	192.168.10.1	192.168.10.174	ICMP	98	Echo (ping) reply id=0x21ec, seq=4/1024, ttl=64 (request in 114)

Рисунок 1.3 – Отображение отправленных командой ping запросов

```
> Frame 89: Packet, 98 bytes on wire (784 bits), 98 bytes cap
> Ethernet II, Src: CudyTechnolo_64:c5:48 (80:af:ca:64:c5:48)
> Internet Protocol Version 4, Src: 192.168.10.1, Dst: 192.16
v Internet Control Message Protocol
    Type: Echo (ping) reply (0)
    Code: 0
    Checksum: 0xd126 [correct]
    [Checksum Status: Good]
    Identifier (BE): 8684 (0x21ec)
    Identifier (LE): 60449 (0xec21)
    Sequence Number (BE): 0 (0x0000)
    Sequence Number (LE): 0 (0x0000)
    [Request frame: 88]
    [Response time: 4.441 ms]
```

Рисунок 1.4 – Задержка одного конкретного запроса в Wireshark

```
64 bytes from 192.168.10.1: icmp_seq=0 ttl=64 time=7.747 ms
```

Рисунок 1.5 – Задержка того же запроса, только в терминале

Можно заметить явную разницу в показателях Wireshark и консоли, это происходит по той причине, что они по-разному считают задержку. Wireshark перехватывает пакет максимально близко к сетевому адаптеру, в то время как команда ping включает также задержки системы для обработки этого пакета.

Задание 2. TRACERT / TRACEROUTE

1. Изучите принципы работы названных утилит: рассмотрите как вариант работы через ICMP, так и через UDP.

Будет рассматриваться traceroute через связку UDP-ICMP.

Traceroute используется для демонстрации всех узлов, пройденных для достижения хоста.

Сама связка выглядит следующим образом: отправляется UDP пакет от устройства на ближайший узел в цепочке, у этого пакета изначально TTL (время жизни) 1, как только он отправляется на ближайший узел, сам узел отнимает от значения единицу и проверяет значение. Если значение 0, то он отправляет ICMP запрос отправителю о том, что время жизни пакета истекло. К слову, отправляется сразу 3 UDP пакета, это необходимо для выявления задержек и потери пакетов.

2. Отфильтруйте в Wireshark трафик по протоколу, используемому вашей версией утилиты (в Windows это tracert, для Linux - traceroute).

Рисунок 2.1 – Фильтрация по конкретному

Стоит добавить, что traceroute использует порты 33434 и выше. Сделано это именно для того, чтобы вдруг порт не совпал с существующим портом узла. С помощью этого знания можно явно понять, что это за запрос и с какой целью он отправлялся.

3. Запустите утилиту над любым узлом глобальной сети.

```
norm — -zsh — 77×24
(base) andreika@MacBook-Pro-Andrej norm % traceroute bsuir.by
traceroute to bsuir.by (134.17.213.43), 64 hops max, 40 byte packets
 1  192.168.10.1 (192.168.10.1)  4.384 ms  2.641 ms  2.329 ms
 2  asr-dc.bn.by (212.98.160.250)  4.968 ms  5.086 ms  4.399 ms
 3  195.137.180.26 (195.137.180.26)  54.766 ms  4.300 ms  3.848 ms
 4  172.24.145.5 (172.24.145.5)  4.773 ms  4.423 ms  5.290 ms
 5  172.24.145.6 (172.24.145.6)  3.891 ms  3.572 ms  2.627 ms
 6  172.24.145.1 (172.24.145.1)  5.167 ms  4.449 ms  4.645 ms
 7  134.17.208.1 (134.17.208.1)  5.028 ms  4.295 ms  4.356 ms
 8  100-210-17-134-dynamic-pool.internet.mts.by (134.17.210.100)  5.354 ms  6
.394 ms  5.786 ms
 9  * *^C
(base) andreika@MacBook-Pro-Andrej norm %
```

Рисунок 2.2 – Traceroute по bsuir.by

Здесь можно увидеть 3 разных значения задержки в каждой строчке – по одному значению на каждый пакет.

4. Соотнесите и прокомментируйте вывод утилиты и перехваченные сообщения.

89	19.678278	192.168.10.174	134.17.213.43	UDP	54	60525 → 33435 Len=12
90	19.680894	192.168.10.1	192.168.10.174	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)

Рисунок 2.3 – Выбран самый первый отправленный пакет. UDP

```
Internet Protocol Version 4, Src: 192.168.10.174, Dst: 134.17.213.43
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 40
    Identification: 0xec6e (60526)
  > 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
  > Time to Live: 1
```

Рисунок 2.4 – Подробности UDP

135	19.874278	192.168.10.174	134.17.213.43	UDP	54	60525 → 33451 Len=12
136	19.878479	172.24.145.1	192.168.10.174	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)

Рисунок 2.5 – Выбран первый из шестой тройки пакетов

```
Internet Protocol Version 4, Src: 192.168.10.174, Dst: 134.17.213.43
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 40
  Identification: 0xec7e (60542)
> 000. .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 6
  Protocol: UDP (17)
```

Рисунок 2.6 – Подробности UDP

Здесь TTL соответствует порядковому номеру ряда в консоли выше. Так что полученные пакеты соответствуют результатам команды traceroute. В сообщении ICMP указано то, что TTL было исчерпано.

Задание 3. ARP

1. Изучите назначение и принцип работы протокола ARP.

Adress Resolution Protocol – протокол, предназначенный определения MAC адреса на основе IP адреса.

Данные о таких соответствиях хранятся в arp-таблице, представленной в виде кэша в оперативной памяти.

Используется следующий алгоритм:

Сперва устройство для нахождения соответствия заглядывает в эту arp-таблицу, если соответствие не найдено, то устройство отправляет широковещательный mac запрос на все устройства в сети с сообщением: «Кто имеет ip: x.x.x.x. – отзовитесь». Когда устройство с таким IP адресом получает этот запрос, то оно отправляет ответный запрос содержащий его mac адрес.

Далее устройство заносит полученные сведения в arp-таблицу.

2. Отобразите ARP-таблицу с помощью утилиты arp.

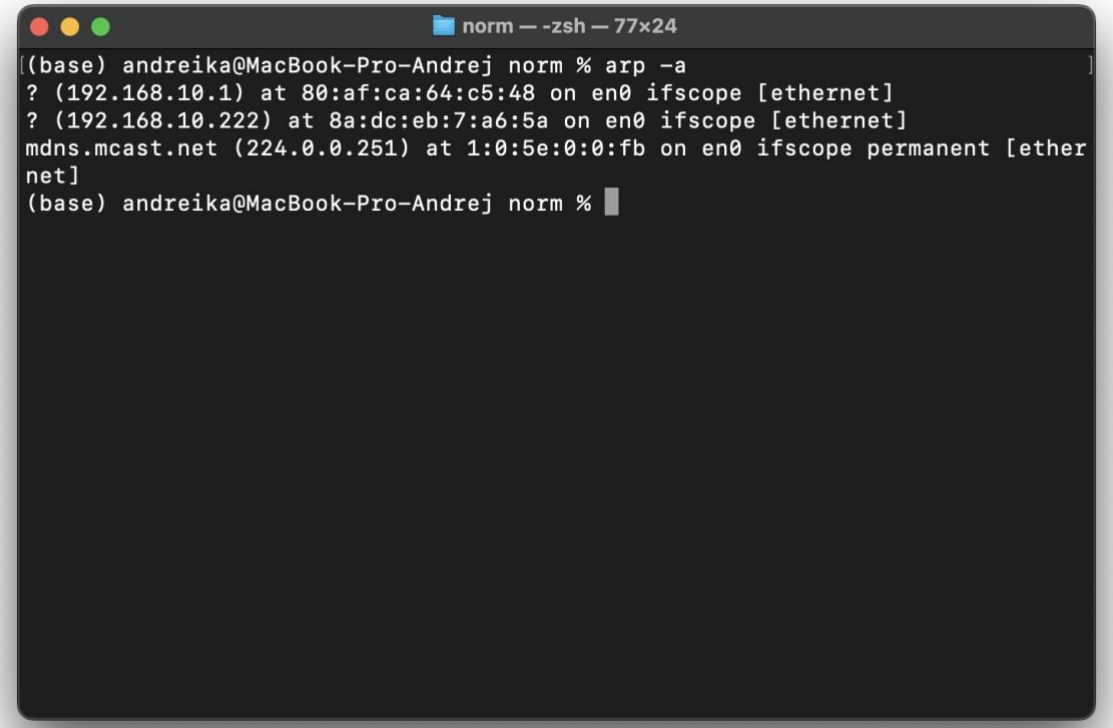


Рисунок 3.1 – Работа команды arp

3. Отфильтруйте в Wireshark ARP-пакеты, отправленные вашему узлу или вашему основному шлюзу (в Windows для определения соответствующих IP-адресов служит утилита ipconfig).

Рисунок 3.1 – Фильтрация ARP по узлу и шлюзу

4. Выполните ping основного шлюза.

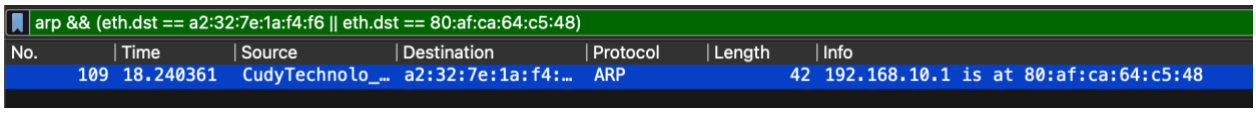


Рисунок 3.2 – Результаты после пинга шлюза

На изображении имеется только ответный запрос, который был получен ещё до пинга шлюза.

5. Прокомментируйте наличие/отсутствие сообщений.

Отсутствие сообщений объясняется тем, что у устройства уже имеются данные в таблице arp, в связи с чем arp запрос не был выполнен.

6. Удалите сопоставление адресов для вашего основного шлюза.

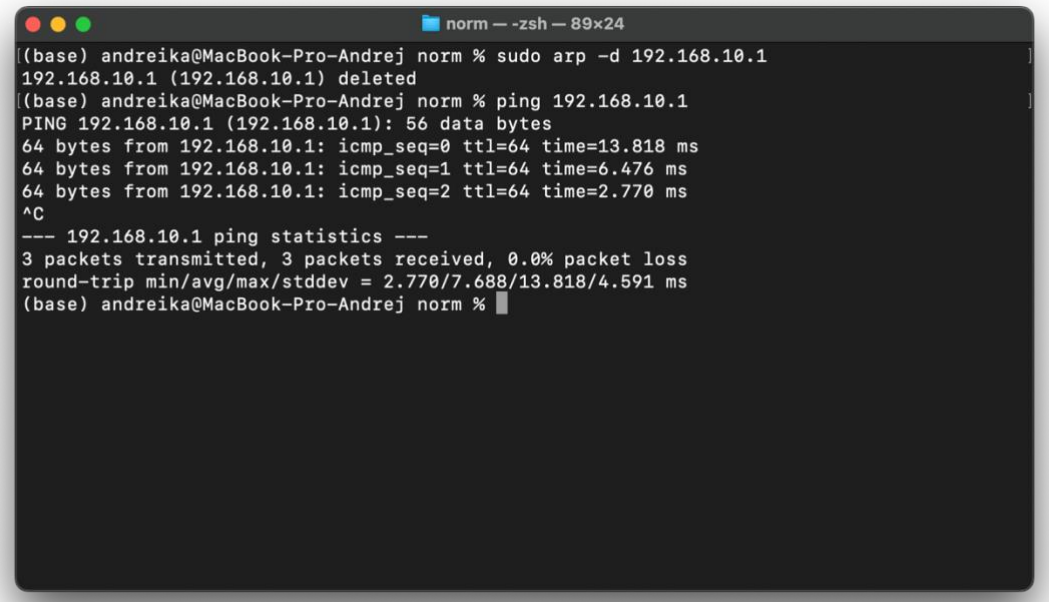


Рисунок 3.3 – Удаление адреса шлюза

7. Прокомментируйте результаты работы Wireshark.

arp

No.	Time	Source	Destination	Protocol	Length	Info
3	0.089639	a2:32:7e:1a:f…	Broadcast	ARP	42	Who has 192.168.10.1? Tell 192.168.10.174
4	0.094424	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	192.168.10.1 is at 80:af:ca:64:c5:48
9	0.321046	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	192.168.10.1 is at 80:af:ca:64:c5:48
51	6.570748	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	Who has 192.168.10.174? Tell 192.168.10.1
52	6.570903	a2:32:7e:1a:f…	CudyTechnolo_64…	ARP	42	192.168.10.174 is at a2:32:7e:1a:f4:f6
315	37.377331	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	Who has 192.168.10.174? Tell 192.168.10.1
316	37.377519	a2:32:7e:1a:f…	CudyTechnolo_64…	ARP	42	192.168.10.174 is at a2:32:7e:1a:f4:f6
484	60.081760	a2:32:7e:1a:f…	Broadcast	ARP	42	Who has 192.168.10.1? Tell 192.168.10.174
485	60.087197	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	192.168.10.1 is at 80:af:ca:64:c5:48
697	73.765867	a2:32:7e:1a:f…	Broadcast	ARP	42	Who has 192.168.10.1? Tell 192.168.10.174
698	73.769977	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	192.168.10.1 is at 80:af:ca:64:c5:48
929	90.318440	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	192.168.10.1 is at 80:af:ca:64:c5:48
1741	174.698914	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	Who has 192.168.10.174? Tell 192.168.10.1
1743	174.699117	a2:32:7e:1a:f…	CudyTechnolo_64…	ARP	42	192.168.10.174 is at a2:32:7e:1a:f4:f6
1796	180.313434	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	192.168.10.1 is at 80:af:ca:64:c5:48
2125	221.999791	a2:32:7e:1a:f…	Broadcast	ARP	42	Who has 192.168.10.1? Tell 192.168.10.174
2126	222.006739	CudyTechnolo_…	a2:32:7e:1a:f4:…	ARP	42	192.168.10.1 is at 80:af:ca:64:c5:48

Рисунок 3.4 – Вывод после удаления шлюза

Так как теперь у устройства не имеется данных об адресе роутера, то он отправляет широковещательный запрос. В случае удаления данных о шлюзе, ARP запрос отправляется.

Объяснение наличия в ARP трафике запросов сразу на целевой MAC: *https://networkengineering.stackexchange.com/a/28807*.

Запрос сразу на целевой адрес выполняется в том случае, когда нужно обновить устаревшие данные о уже существовавшем узле в сети для уточнения его ip адреса.

Задание 4. DNS

1. Изучите службу DNS и соответствующий протокол.

Domain Name System – нужен для получения адреса сайта по его домену.

Сперва проверяется наличие адреса по этому домену в кэше (может храниться в браузере). Если ничего не найдено, то отправляется dns запрос на порт 53. В этом запросе находится вопрос о том, на каком адресе находится сайт с конкретным доменом. Сам протокол, в котором хранится сообщение находится на прикладном уровне.

В свою очередь служба состоит из резолвера (клиентской части) – программы, отправляющей запрос, и DNS-сервера, который этот запрос принимает. По умолчанию, если отправлять DNS запрос и ничего не настраивать, то будет этот запрос будет отправляться на DNS-сервер, который был назначен изначально с помощью DHCP.

No.	Time	Source	Destination	Protocol	Length	Info
38	1.403008	192.168.10.174	192.168.10.1	DNS	68	Standard query 0xbf8d A bsuir.by
39	1.407399	192.168.10.1	192.168.10.174	DNS	84	Standard query response 0xbf8d A bsuir.by A 134.17.213.43

Рисунок 4.1 – Пример DNS запроса без настройки

Если воспользоваться ручной настройкой, и выбрать сервер самостоятельно указав желаемый сервер в команде traceroute:

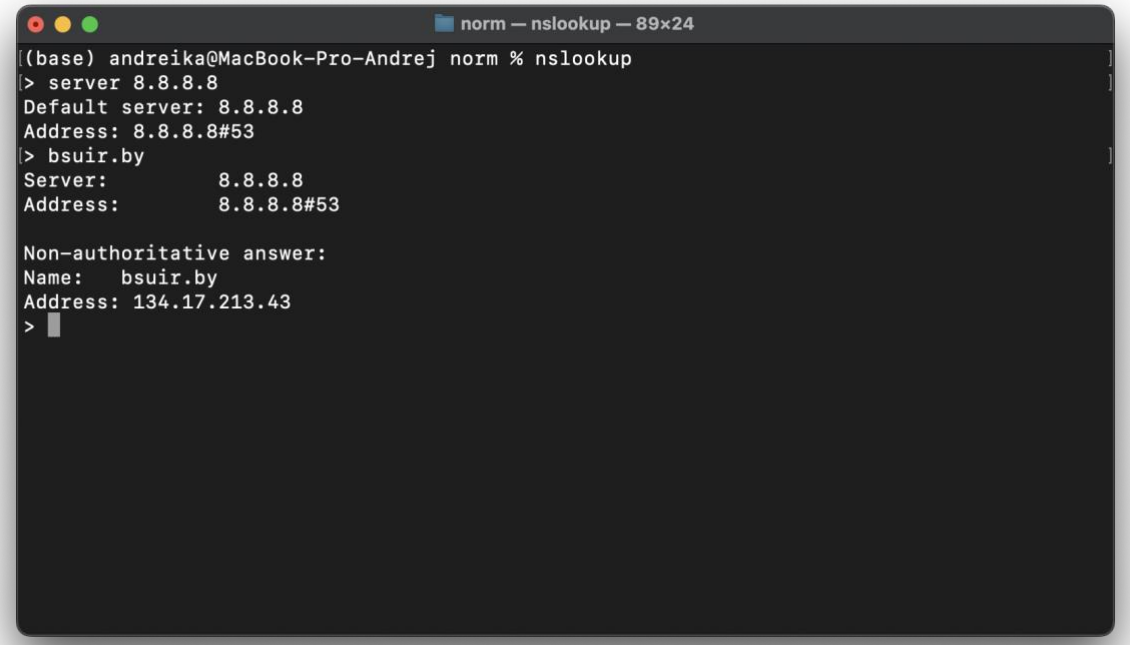


Рисунок 4.2 – Создание DNS запроса с выбором сервера

Будет получен следующий результат:

No.	Time	Source	Destination	Protocol	Length	Info
256	20.188481	192.168.10.174	8.8.8.8	DNS	68	Standard query 0x57ef A bsuir.by
257	20.202059	8.8.8.8	192.168.10.174	DNS	84	Standard query response 0x57ef A bsuir.by A 134.17.213.43

Рисунок 4.3 – Тот же самый DNS запрос, только в Wireshark

2. Отфильтруйте в Wireshark DNS-трафик.

Рисунок 4.4 – Фильтрация по DNS

3. С помощью журнала Wireshark покажите, какой протокол транспортного уровня используется для работы DNS.

```
> Frame 38: Packet, 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface en0, id 0
> Ethernet II, Src: a2:32:7e:1a:f4:f6 (a2:32:7e:1a:f4:f6), Dst: CudyTechnolo_64:c5:48 (80:af:ca:64:c5:48)
> Internet Protocol Version 4, Src: 192.168.10.174, Dst: 192.168.10.1
> User Datagram Protocol, Src Port: 58617, Dst Port: 53
> Domain Name System (query)
```

Рисунок 4.5 – Подробности DNS-запроса

Используется протокол UDP. Так как это слишком простой запрос, в котором потеря пакета будет не страшна.

4. Выполните DNS запрос к Google DNS (8.8.8.8) вручную с помощью утилиты *nslookup*.

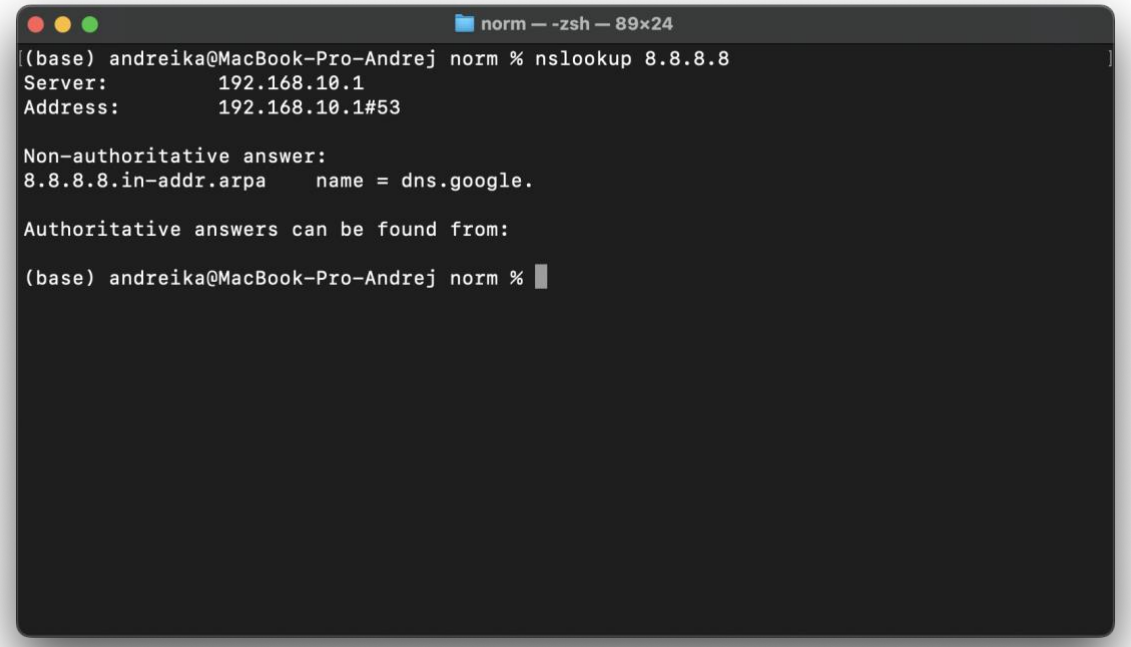
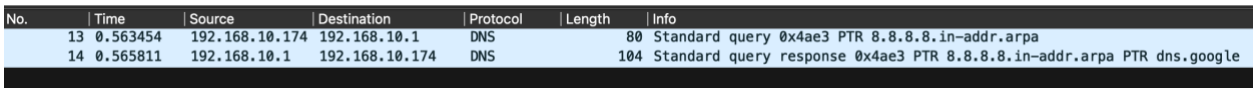


Рисунок 4.6 – Выполнение DNS запроса на dns.google

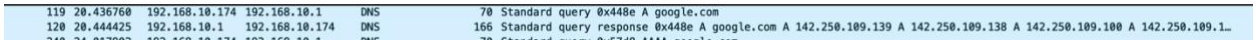


No.	Time	Source	Destination	Protocol	Length	Info
13	0.563454	192.168.10.174	192.168.10.1	DNS	80	Standard query 0x4ae3 PTR 8.8.8.8.in-addr.arpa
14	0.565811	192.168.10.1	192.168.10.174	DNS	104	Standard query response 0x4ae3 PTR 8.8.8.8.in-addr.arpa PTR dns.google

Рисунок 4.7 – Результаты запроса

5. Прокомментируйте результаты работы Wireshark (обратите внимание на типы DNS записей: A и AAAA).

При попытке отправить стандарнтый DNS запрос, будет получен IPv4-адрес.



119	20.436760	192.168.10.174	192.168.10.1	DNS	70	Standard query 0x448e A google.com
120	20.444425	192.168.10.1	192.168.10.174	DNS	166	Standard query response 0x448e A google.com A 142.250.109.139 A 142.250.109.138 A 142.250.109.100 A 142.250.109.1…

Рисунок 4.8 – DNS типа A

```
norm — -zsh — 89×24
(base) andreika@MacBook-Pro-Andrej norm % nslookup -type=AAAA google.com
Server:         192.168.10.1
Address:        192.168.10.1#53

Non-authoritative answer:
google.com      has AAAA address 2a00:1450:4025:800::71
google.com      has AAAA address 2a00:1450:4025:800::65
google.com      has AAAA address 2a00:1450:4025:800::64
google.com      has AAAA address 2a00:1450:4025:800::8a

Authoritative answers can be found from:

(base) andreika@MacBook-Pro-Andrej norm %
```

Рисунок 4.9 – Создание запроса с заданным типом AAAA

При отправке DNS-запроса с выбранным типом AAAA, будет отправлен AAAA запрос, соответственно будут получены IPv6-адреса.

2	0.022277	192.168.10.174	192.168.10.1	DNS	70	Standard query 0xfbc6 AAAA google.com
3	0.026262	192.168.10.1	192.168.10.174	DNS	182	Standard query response 0xfbc6 AAAA google.com

Рисунок 4.10 – Запрос типа AAAA

6. С помощью утилиты ping выполните обратный DNS-запрос, определив доменное имя адреса 8.8.8.8.

Так как на Mac os команда ping не показывает доменные имена, то будет использована команда host.

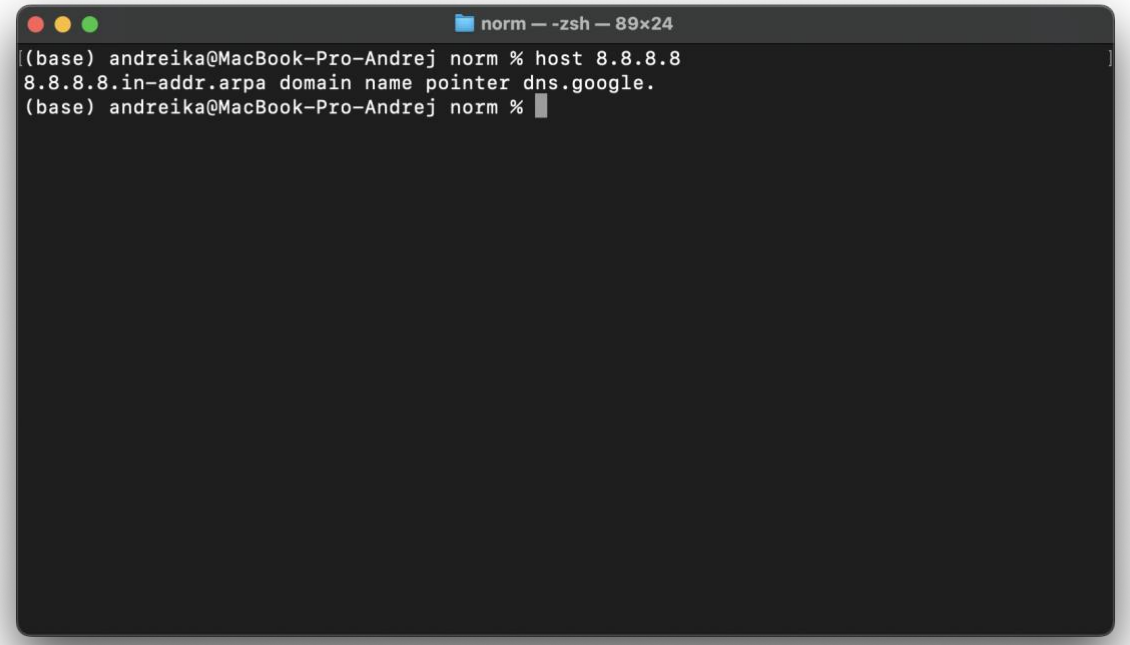
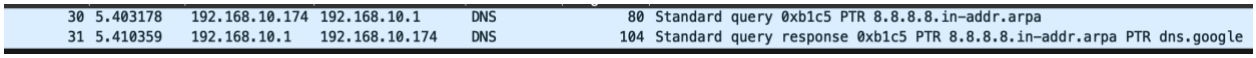


Рисунок 4.11 – Запуск команды host

7. Прокомментируйте результаты работы Wireshark.



30	5.403178	192.168.10.174	192.168.10.1	DNS	80	Standard query 0xb1c5 PTR 8.8.8.8.in-addr.arpa
31	5.410359	192.168.10.1	192.168.10.174	DNS	104	Standard query response 0xb1c5 PTR 8.8.8.8.in-addr.arpa PTR dns.google

Рисунок 4.12 – Результат команды host

Команда host использует обратный DNS-запрос, а именно запрос типа PTR, который исходя из данного IP адреса выдает соответствующее доменное имя.

8. Отобразите кэш DNS (в Windows это можно сделать с помощью *ipconfig*).

```
andreika — -zsh — 104×24
(base) andreika@MacBook-Pro-Andrej ~ % time dscacheutil -q host -a name google.com

name: google.com
ipv6_address: 2a00:1450:4025:800::66
ipv6_address: 2a00:1450:4025:800::64
ipv6_address: 2a00:1450:4025:800::8b
ipv6_address: 2a00:1450:4025:800::71

name: google.com
ip_address: 142.250.109.100
ip_address: 142.250.109.113
ip_address: 142.250.109.138
ip_address: 142.250.109.101
ip_address: 142.250.109.102
ip_address: 142.250.109.139

dscacheutil -q host -a name google.com  0.00s user 0.00s system 48% cpu 0.020 total
(base) andreika@MacBook-Pro-Andrej ~ %
```

Рисунок 4.13 – Отображение кэша

9. Очистите кэш DNS (в Windows это можно сделать с помощью *ipconfig*).

```
andreika — -zsh — 104×24
(base) andreika@MacBook-Pro-Andrej ~ % sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

Рисунок 4.14 – Очистка кэша

10. Повторите запрос, выполненный в пункте 4.

Для наглядности в этот раз будет использоваться команда ping, так как nslookup игнорирует кэш и сразу отправляет сетевой запрос.

```
andreika — -zsh — 104×24
(base) andreika@MacBook-Pro-Andrej ~ % sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder

(base) andreika@MacBook-Pro-Andrej ~ % ping -c 1 dns.google
PING dns.google (8.8.4.4): 56 data bytes
64 bytes from 8.8.4.4: icmp_seq=0 ttl=118 time=12.015 ms

--- dns.google ping statistics ---
1 packets transmitted, 1 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 12.015/12.015/12.015/0.000 ms
(base) andreika@MacBook-Pro-Andrej ~ % ping -c 1 dns.google
PING dns.google (8.8.4.4): 56 data bytes
64 bytes from 8.8.4.4: icmp_seq=0 ttl=118 time=18.088 ms

--- dns.google ping statistics ---
1 packets transmitted, 1 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 18.088/18.088/18.088/nan ms
(base) andreika@MacBook-Pro-Andrej ~ %
```

Рисунок 4.15 – Очистка кэша и выполнение команды ping дважды

11. Прокомментируйте результаты работы Wireshark и содержимое кэша.

Было выполнено две ping команды, в первом случае, как можно увидеть в wireshark, был отправлен DNS-запрос для нахождения адреса домена.

При втором использовании таких запросов не наблюдается. Такое произошло по той причине, что кэш был сохранен в систему и ping не понадобилось отправлять запрос, чтобы узнать адрес, так как он уже известен в кэше.

30	1.953482	192.168.10.174	192.168.10.1	DNS	70	Standard query 0xfb20 A dns.google
31	1.956015	192.168.10.1	192.168.10.174	DNS	102	Standard query response 0xfb20 A dns.google A 8.8.4.4 A 8.8.8.8

Рисунок 4.16 – Результат выполнения команды ping дважды

Задание 5. IPCONFIG/IFCONFIG

Выполните команду *ipconfig* (или *ifconfig* для linux/macOS) и прокомментируйте
выведенные сетевые адаптеры и их параметры (физический адрес, IP-адрес, шлюз, маска подсети).

lo0 (loopback) — виртуальный интерфейс для внутренних нужд системы. Позволяет компьютеру обращаться к самому себе по адресу 127.0.0.1.

en0 (Wi-Fi) — основной сетевой адаптер, статус active. Имеет физический адрес a2:32:7e:1a:f4:f6, локальный IP-адрес 192.168.10.174 и маску подсети 255.255.255.0. Через него идет основной интернет-трафик.

awdl0 (Apple Wireless Direct Link) — служебный интерфейс для связи устройств Apple между собой напрямую. Используется для работы AirDrop и Sidecar, имеет MAC-адрес 8a:84:76:02:41:70.

bridge0 — виртуальный мост, объединяющий интерфейсы en1, en2 и en3. Необходим для корректной работы и взаимодействия портов Thunderbolt.

utun0-utun5 — виртуальные туннельные интерфейсы для работы VPN-соединений и сервисов шифрования трафика.

en1-en6, anpi0-anpi2 — дополнительные физические и системные интерфейсы, на данный момент не задействованы (status: inactive).

Контрольные вопросы

- Как отличается процесс передачи данных между узлами, когда оба узла находятся в одной локальной сети, и когда в разных?

Когда два узла находятся в одной сети, то данные передаются по канальному уровню, то есть напрямую по mac адресам. В случае, когда узлы находятся в разных сетях, данные передаются по сетевому уровню, то есть упаковываются в пакеты и работают уже по другому механизму.

- Для чего используется маска подсети?

Так как существуют глобальные и локальные IP адреса, то для удобства у локальных адресов имеется примерно один и тот же ip адрес. Выглядит это примерно следующим образом: 192.168.10.x – где x – отвечает за идентификацию устройства в локальной сети. Группа значения до x – маска сети. Для большего числа устройств, оно может выглядеть так: 192.168.x.x.

- В скольких сетях находится ваш компьютер? Сколько у него IP-адресов? А MAC-адресов?

Исходя из данных ifconfig, мой компьютер подключен к одной сети, имеет en0 ip адреса, так же имеет петлевой ip адрес. А так же много mac адресов для разных систем.

- Для чего в ARP-запросе отправитель указывает свой IP-адрес?

Чтобы устройство получатель тоже могло сопоставить ip адрес с mac адресом сразу же и не отправлять дополнительный arp запрос.

- Можно в настройках системы в качестве адреса DNS-сервера указать доменное имя dns.google вместо его IP-адреса 8.8.8.8? Почему?

Нет, нельзя. Так как для того, чтобы узнать адрес DNS-сервера dns.google нужно быть подключенным к dns серверу, а так как DNS-сервер был указан dns.google, то это не является возможным, так как происходит цикл обращения к dns серверу, адрес которого неизвестен.

- Для чего в IP-пакетах служит TTL?

Для предотвращения случаев, когда у пакета происходят ошибки маршрутизации. В таком случае TTL предотвращает вечное блуждание пакетов по узлам. То есть каждый узел отнимает по одному значению TTL и, если это значение стало 0, то пакет уничтожается.